



FitnessTracker SaaS Enterprise AI Systems & Production-Grade Architecture

A deep-dive technical blueprint mapping multi-tier subscription systems, Groq LLM rate-limit gates, split CORS/COOP setups, and custom visual React design systems.

Author: Hussein A-H

Repository: HusseinA-H/FitnessTracker

Production Build Status: Verified 100% Green / Zero Warnings

1. Executive Summary & Product Vision

FitnessTracker is a premium, enterprise-ready software-as-a-service (SaaS) platform offering highly tailored, data-driven fitness coaching, progress analytics, and diet planning. Combining a modular Django REST Framework (DRF) backend with a reactive React SPA, the platform is built from the ground up for high concurrency, premium aesthetics, and rigid security.

The **Product Vision** centers around bridging the gap between automated telemetry (weight and body metrics tracking) and high-context personal guidance. By leveraging state-of-the-art Large Language Models (LLMs) via Groq Cloud's ultra-fast LPU inference, the app provides real-time fitness analysis that automatically locks past telemetry metrics to protect consistency. Billing checkouts are completely integrated via Cairo Nile Instapay screenshot uploads, approved and activated in single-click actions inside a restricted admin console.

Core Architecture Goal: Disconnect developer environments completely from production assets using multi-stage Docker wrappers, environment setting splits, and strict CORS/COOP separation layers.

2. SaaS Business Model & Subscription System

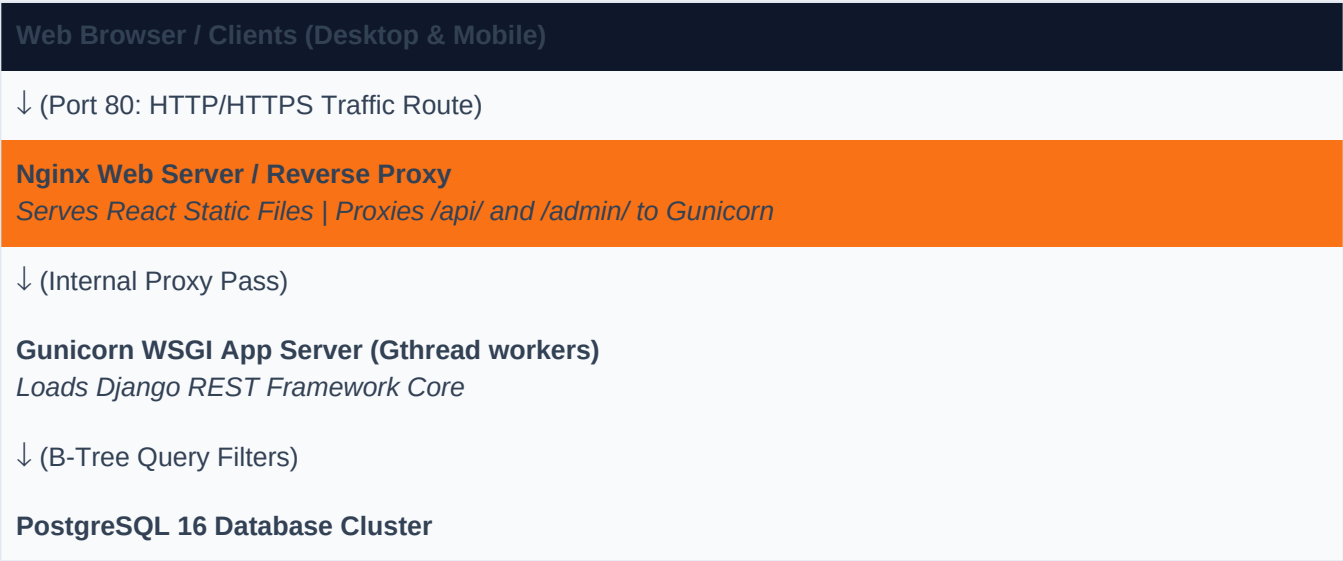
The business architecture operates on a multi-tier recurring membership model. Capabilities, workout thresholds, and AI tokens quotas are gated dynamically based on the active plan tier caching record on the user object.

Plan Tier	Price / Duration	AI Limit (Prompts/Mo)	Workout Logs Limit	Dashboard Analytics
Basic	\$9.99 / 30 Days	10 Prompts	50 Logs	Standard
Premium	\$19.99 / 30 Days	50 Prompts	Unlimited	Advanced
Elite	\$39.99 / 90 Days	200 Prompts	Unlimited	Advanced + Graphs

Self-Checkout & Manual Billing Approvals: Users select a membership tier and initiate a self-checkout where they upload an Instapay/bank transfer receipt screenshot. The backend logs the transaction as PENDING_APPROVAL. Administrators review the receipts inside the Admin ledger, verifying the amount and details before clicking **Approve**. This activates the plan and updates the user's cached subscription pointer immediately.

3. Technical System Architecture

The system employs a segregated multi-container configuration to handle loads dynamically, structured cleanly under docker-compose.yml. The React SPA is bundled in multi-stage builds and served via Nginx, which also acts as a reverse proxy routing requests under /api/ and /admin/ paths to Gunicorn.



Frontend Architecture (React 19 & Vite): The interface runs as a client-side Single Page Application (SPA). Global authentication session scopes are managed via Zustand stores. Pages are lazy-loaded dynamically under AppRoutes.jsx to optimize browser initial load speeds, wrapped with Framer Motion AnimatePresence hooks for visual transitions.

Backend Architecture (DRF): To keep Python imports clean, modules are organized inside `backend/apps/`. Configurations are environment-separated via `base.py`, `local.py`, and `production.py` settings. The database configuration strictly requires PostgreSQL to preserve strict testing-to-production database engine parity.

4. Database Design & API Schema

The relational database schema is configured in PostgreSQL with UUID primary keys on all tables. Composite indexes and B-trees optimize heavy queries across active user, logging, and AI records.

Table	Key Columns & Types	Constraints / Indexes
users_user	role: VARCHAR(20) current_sub_id: UUID	PRIMARY: UUID INDEX: email, role
workouts_workout_log	weight: DECIMAL(6,2) is_locked: BOOLEAN	INDEX: (user, exercise, date) INDEX: is_locked

subscriptions_subscription	status: VARCHAR(30) is_approved: BOOLEAN	INDEX: (user, status, approved)
ai_ai_usage	total_tokens: INT estimated_cost: DEC(10,6)	INDEX: (user, date, type)

5. AI Integration & Security Systems

FitnessTracker features advanced integrations with Groq Cloud for fitness analysis. Responses stream dynamically using LLM prompts run over LPUs.

AI Security & Prompt Sanitization: All client prompts undergo validation in ai/utils.py before processing. The sanitization layer:

- Enforces a 2000-character maximum length limit.
- Strips all HTML/XML tags.
- Applies 9 regex validation checks to identify and block jailbreak attempts.
- Logs blocked malicious inputs with `SUSPICIOUS\_AI\_PROMPT\_BLOCKED` audit actions.

Model Profiles & Pricing Breakdown: The backend switches models based on speed or quality priorities:

Model Name	API Identifier	Key Use Case	Estimated Cost (per 1M Tok)
Llama 3.3 70B	llama-3.3-70b-versatile	Default Chat & Insights	Input: \$0.59 / Output: \$0.79
Llama 3.1 8B	llama-3.1-8b-instant	Fast Replies & Fallbacks	Input: \$0.05 / Output: \$0.08
Mixtral 8x7B	mixtral-8x7b-32768	Long Thread Dialogues	Input: \$0.24 / Output: \$0.24

6. DevOps Setup & Performance Architecture

1. CORS & COOP Headers Split Resolution:

Cross-origin POST requests failed on initial deployment due to Django's default `SECURE_CROSS_ORIGIN_OPENER_POLICY` setting. This injected COOP: same-origin on all responses, blocking cross-origin fetch requests in modern browsers when combined with cookies credentials.

Solution: Disabled default COOP headers in settings (`SECURE_CROSS_ORIGIN_OPENER_POLICY = None`), and configured a custom `SecurityHeadersMiddleware`. This middleware selectively skips CSP/COOP injections on API routes (under `/api/`) while keeping it active for HTML pages like the Django Admin site.

2. Production Hardening & Caching:

- ****Database Optimization**:** Utilizes B-tree indexing and selective prefetching (`select_related` and `prefetch_related`) to prevent N+1 database queries.

- ****Docker Optimization****: Configured Gunicorn with 3 concurrent workers, 2 threads, and a 120s timeout to process Groq's long-running AI requests without timeout failures.
- ****Frontend Speed****: Implemented lazy routing chunks and local assets caching for high speed.

7. Lessons Learned & Future Roadmap

Lessons Learned:

1. ****Middleware Pipeline Flow****: Understanding the reverse-order processing flow in Django middleware is critical. Changes applied in early middlewares can be overwritten by downstream middlewares during response building.
2. ****Strict Environment Parity****: Eliminating SQLite completely from dev workflows avoided query syntax mismatches and dialect-specific behavior during deployment.

Future Roadmap:

- ****Gateway Integration****: Integrate direct automated payment processing via Stripe Elements.
- ****WebSockets Support****: Refactor the AI Chat interface to use channels/websockets for fast stream completion.
- ****Offline Telemetry App****: Deploy a Service Worker wrapper for offline workout metrics logging.